

3B2 FPGA Programming Lab Report

Liu Zihe^{1*}

¹University of Cambridge

Abstract

This report documents the design, implementation and testing of a traffic light controller on an Altera Cyclone V FPGA. A three-state finite state machine controlling a pedestrian crossing is described in Verilog, compiled, and configured onto a DE1-SoC board via JTAG, and is improved with a vehicle lockout.

1 Traffic Light Controller Design

We design a controller for a pedestrian light-controlled crossing, satisfying the following requirements:

1. The initial state is vehicle GREEN and pedestrian RED.
2. State changes occur only on demand: a pedestrian request or a reset.
3. After a request, the sequence is: vehicle YELLOW for 5 s, then vehicle RED with pedestrian GREEN for 10 s, then back to the initial state.
4. An asynchronous reset returns to the initial state from any state.

These requirements are captured by a three-state FSM shown in Figure 1. State G is the idle state with vehicle GREEN and pedestrian RED. A pedestrian request triggers a transition to state Y (vehicle YELLOW), which serves as a warning to vehicles. After a 5 s timeout the FSM enters state R (vehicle RED, pedestrian GREEN), allowing pedestrians to cross. A 10 s timeout then returns the FSM to state G. An asynchronous active-low reset forces the FSM back to state G from any state.

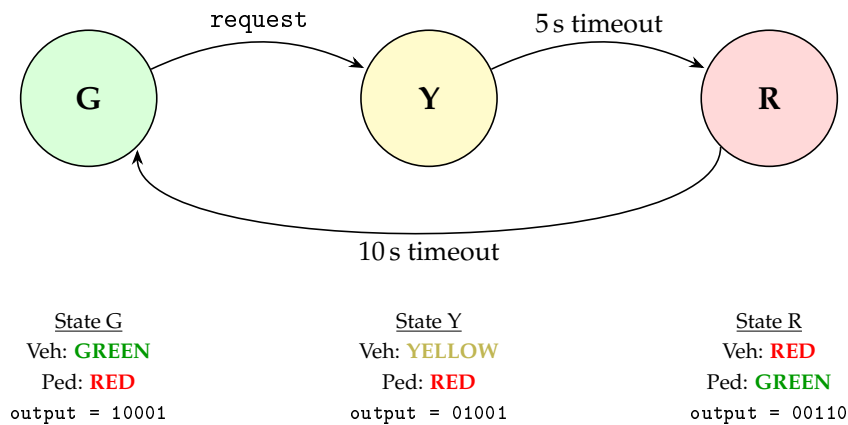


Figure 1: State diagram for the traffic light controller FSM. The 5-bit output vector is {veh_G, veh_Y, veh_R, ped_G, ped_R}. reset (active LOW, asynchronous) returns to state G from any state.

The outputs of each state and the transitions to the next states, are summarised in Table 1. Each state activates exactly one vehicle light and one pedestrian light.

Table 1: State table for the traffic light controller.

State	Next State				Outputs				
	reset	request	5 s	10 s	veh_G	veh_Y	veh_R	ped_G	ped_R
G	G	Y	—	—	1	0	0	0	1
Y	G	—	R	—	0	1	0	0	1
R	G	—	—	G	0	0	1	1	0

*CRSID: zl559. Date: 20th February 2026

2 Verilog Implementation

Listing 1: Basic traffic light controller.

```
1 // =====
2 // Traffic Light Controller (TLC) for Pedestrian Crossing
3 // =====
4 // Implements a 3-state FSM for a pedestrian light-controlled crossing.
5 //
6 // FSM states:
7 //   State G (initial) - vehicle GREEN, pedestrian RED
8 //   State Y           - vehicle YELLOW, pedestrian RED (transitional)
9 //   State R           - vehicle RED, pedestrian GREEN
10 //
11 // Behaviour:
12 //   - Initial state: State G (vehicle GREEN, pedestrian RED)
13 //   - On pedestrian request: State G -> State Y (5s) -> State R (10s) -> State G
14 //   - Reset returns to State G at any time
15 //
16 // Output mapping: {veh_GREEN, veh_YELLOW, veh_RED, ped_GREEN, ped_RED}
17 // =====
18
19 module tlc (
20     input wire clk, // 50 MHz on-board clock (PIN_AF14)
21     input wire request, // Pedestrian push-button (active LOW on DE1-SoC)
22     input wire reset, // Reset push-button (active LOW, asynchronous)
23     output reg [4:0] \output // 5-bit output: {veh_GREEN, veh_YELLOW, veh_RED, ped_GREEN
24         , ped_RED}
25 );
26
27 // --- State encoding (2-bit) ---
28 // Only 3 states needed, so 2 bits suffice
29 localparam G = 2'd0; // State G: vehicle GREEN / pedestrian RED
30 localparam Y = 2'd1; // State Y: vehicle YELLOW / pedestrian RED
31 localparam R = 2'd2; // State R: vehicle RED / pedestrian GREEN
32
33 reg [1:0] state; // Current FSM state
34 reg [28:0] count; // 29-bit counter for timing delays
35 // 29 bits can count up to  $2^{29} - 1 = 536,870,911$ 
36 // which is enough for 500,000,000 (10s @ 50 MHz)
37
38 // =====
39 // Sequential logic: state transitions & timer
40 // =====
41 // Triggered on rising clock edge OR falling edge of reset (async reset).
42 // DE1-SoC push-buttons are active LOW, so reset triggers on negedge.
43 always @(posedge clk or negedge reset) begin
44     if (!reset) begin
45         // --- Asynchronous reset: return to State G ---
46         state <= G;
47         count <= 29'd0;
48     end else begin
49         case (state)
50
51             // --- State G: wait for pedestrian request ---
52             // request is active LOW (button pressed = 0), so we check == 0.
53             // No timer; we stay in State G indefinitely until request.
54             G: begin
55                 if (request == 1'b0) begin
56                     state <= Y;
57                     count <= 29'd0;
58                 end
59             end
60
61             // --- State Y: vehicle YELLOW for 5 seconds ---
62             // 5 seconds @ 50 MHz = 250,000,000 clock cycles.
63             Y: begin
64                 if (count == 29'd250_000_000) begin
65                     state <= R;
66                     count <= 29'd0;
67                 end else begin
68                     count <= count + 29'd1;
69                 end
70             end
71
72             // --- State R: vehicle RED, pedestrian GREEN ---
73             // 10 seconds @ 50 MHz = 500,000,000 clock cycles.
74             R: begin
75                 if (count == 29'd500_000_000) begin
76                     state <= G;
77                     count <= 29'd0;
78                 end else begin
79                     count <= count + 29'd1;
80                 end
81             end
82         endcase
83     end
84 end
```

```

70
71 // --- State R: vehicle RED, pedestrian GREEN for 10 seconds ---
72 // 10 seconds @ 50 MHz = 500,000,000 clock cycles.
73 R: begin
74     if (count == 29'd500_000_000) begin
75         state <= G;
76         count <= 29'd0;
77     end else begin
78         count <= count + 29'd1;
79     end
80 end
81
82 // --- Default: safety fallback to State G ---
83 // Catches undefined states (e.g. state = 2'd3)
84 default: begin
85     state <= G;
86     count <= 29'd0;
87 end
88 endcase
89 end
90 end
91
92 // =====
93 // Combinational logic: output assignment
94 // =====
95 // Maps each state to the 5-bit LED output vector:
96 //   Bit [4] = veh_GREEN
97 //   Bit [3] = veh_YELLOW
98 //   Bit [2] = veh_RED
99 //   Bit [1] = ped_GREEN
100 //   Bit [0] = ped_RED
101 //
102 //   State G: veh_GREEN on,  ped_RED on   -> 10001
103 //   State Y: veh_YELLOW on, ped_RED on   -> 01001
104 //   State R: veh_RED on,   ped_GREEN on  -> 00110
105 always @(*) begin
106     case (state)
107         G:      \output = 5'b10001;
108         Y:      \output = 5'b01001;
109         R:      \output = 5'b00110;
110         default: \output = 5'b10001; // Default to State G
111     endcase
112 end
113
114 endmodule

```

3 Compilation Results

Table 2: Compiler Flow Summary.

Resource	Count
Logic Elements	TODO
Registers	TODO
Pins	TODO

Table 3: Pin assignments for the DE1-SoC board.

Signal	Pin	Component
clk	PIN_AF14	50 MHz Clock
request	PIN_AA15	KEY1 (Push-button)
reset	PIN_AA14	KEY0 (Push-button)
output[0]	PIN_V16	LEDR0 (ped RED)
output[1]	PIN_V17	LEDR2 (ped GREEN)
output[2]	PIN_W17	LEDR4 (veh RED)
output[3]	PIN_Y19	LEDR6 (veh YELLOW)
output[4]	PIN_W21	LEDR8 (veh GREEN)

Table 4: Board testing results.

Initial State	Input Applied	Expected Next State	Pass/Fail
G	request	Y (5 s timer)	TODO
Y	5 s timeout	R (10 s timer)	TODO
R	10 s timeout	G	TODO
Any	reset	G	TODO

4 Pin Assignment

5 Testing

6 Improved Circuit

6.1 Updated State Machine

6.2 Verilog Implementation

6.3 Testing

7 Conclusions

A Appendix

A.1 Testbench

Listing 2: Testbench for the basic TLC.

```

1 // Testbench for basic Traffic Light Controller (tlc.v)
2 // Uses force/release on internal counter to skip long timer waits.
3 `timescale 1ns / 1ps
4
5 module tlc_tb;
6
7     reg clk, request, reset;
8     wire [4:0] out;
9
10    tlc uut (.clk(clk), .request(request), .reset(reset), .\output (out));
11
12    initial clk = 0;
13    always #10 clk = ~clk; // 50 MHz
14
15    localparam G = 2'd0, Y = 2'd1, R = 2'd2;
16    localparam OUT_G = 5'b10001, OUT_Y = 5'b01001, OUT_R = 5'b00110;
17    localparam Y_THRESH = 29'd250_000_000, R_THRESH = 29'd500_000_000;
18
19    integer pass_count = 0, fail_count = 0;
20
21    task check(input [63:0] name, input cond);
22        begin
23            if (cond) begin $display("PASS: %0s", name); pass_count = pass_count + 1; end

```

```

24         else           begin $display("\nFAIL: %0s", name); fail_count = fail_count + 1; end
25     end
26 endtask
27
28 task wait_clks(input integer n);
29     integer i;
30     begin for (i = 0; i < n; i = i + 1) @(posedge clk); end
31 endtask
32
33 // Skip counter to near threshold, then let it trigger naturally
34 task skip_counter(input [28:0] thresh);
35     begin
36         force uut.count = thresh - 2;
37         wait_clks(1);
38         release uut.count;
39         wait_clks(5);
40     end
41 endtask
42
43 initial begin
44     $dumpfile("tlc_tb.vcd");
45     $dumpvars(0, tlc_tb);
46     request = 1; reset = 1;
47
48     // T1: Reset -> state G
49     $display("\n---T1: Reset---");
50     reset = 0; wait_clks(3);
51     check("state=G", uut.state == G);
52     check("out=10001", out == OUT_G);
53     reset = 1; wait_clks(2);
54
55     // T2: Full cycle G -> Y -> R -> G
56     $display("\n---T2: Full cycle---");
57     request = 0; wait_clks(2); request = 1;
58     check("G->Y", uut.state == Y);
59     check("out=01001", out == OUT_Y);
60     skip_counter(Y_THRESH);
61     check("Y->R", uut.state == R);
62     check("out=00110", out == OUT_R);
63     skip_counter(R_THRESH);
64     check("R->G", uut.state == G);
65     check("out=10001", out == OUT_G);
66
67     // T3: Async reset from Y and R
68     $display("\n---T3: Async reset---");
69     request = 0; wait_clks(2); request = 1;
70     check("in_Y", uut.state == Y);
71     reset = 0; wait_clks(2);
72     check("Y_reset->G", uut.state == G);
73     reset = 1; wait_clks(2);
74     // Now reset from R
75     request = 0; wait_clks(2); request = 1; wait_clks(1);
76     skip_counter(Y_THRESH);
77     check("in_R", uut.state == R);
78     reset = 0; wait_clks(2);
79     check("R_reset->G", uut.state == G);
80     reset = 1; wait_clks(2);
81
82     // T4: Idle stability
83     $display("\n---T4: Idle---");
84     request = 1; wait_clks(100);
85     check("stays_G", uut.state == G);
86
87     // Summary
88     $display("\n== %0d passed, %0d failed ==", pass_count, fail_count);
89     if (fail_count == 0) $display("ALL TESTS PASSED");
90     else $display("SOME TESTS FAILED");
91     $finish;
92 end
93
94 endmodule

```